“But Why?” — Data and Query Provenance for Explainable Movie Recommendations

Author: Qianyi Chen (Alfred) · Solo · CSDS 234 · Prof. Yinghui Wu · Case Western Reserve University

Abstract

Modern collaborative-filtering (CF) recommender systems are accurate but opaque: they cannot explain *why* a particular movie was recommended. Such opacity is increasingly a regulatory liability (e.g., GDPR Article 22’s “right to explanation”) and a product-trust concern. This paper applies the database concept of **why-provenance** to a CF recommender on the MovieLens 100K dataset. We define two query classes — *why-provenance* (find the minimal set of ratings whose removal would drop a movie out of top-*k*) and *query rewrite* (find the minimal rating edit that surfaces a desired-but-missing movie) — and design an algorithm-plus-index that answers them efficiently. The core idea is an **inverted index** mapping each movie to its top-*C* contributing raters, enabling pruning from O(|R|) to O(*C*) per query. We evaluate on 60 random (user, movie) pairs across two factors: top-*k* size $k \in \{5, 10, 20\}$ and movie popularity (cold/medium/hot raters). Our index-pruned WhyProv achieves **1.77× average speedup** at $k = 10$ and up to **2.45× at $k = 20$** over a naive full-scan baseline, with the speedup growing monotonically in *k*. The approach is general — it answers the query class for any user, any movie, and any *k* in the tested range.

1. Introduction

1.1 Motivation: “Why was this recommended?”

Imagine a user, Alfred, opens his streaming service and is recommended *Fargo*. Why? A modern CF model — typically matrix factorization such as SVD — can produce the recommendation, but its decision is encoded in 50-dimensional latent vectors that resist human interpretation. Alfred has no way to know whether *Fargo* was suggested because of his love for *Pulp Fiction*, his rating of *The Shawshank Redemption*, or some implicit pattern shared with thousands of strangers.

This question — “*why* was this recommended?” — has become both a regulatory and a product concern. The European Union’s GDPR Article 22 grants users a right to “meaningful information about the logic involved” in automated decisions. From a product perspective, explainable recommendations build user trust, support content discovery, and enable creators to understand reach. Yet most explanation methods today either modify the recommender model (hurting accuracy) or generate post-hoc natural-language rationalizations that may not faithfully reflect the model’s actual decision.

We take a different approach: explain CF recommendations using **data provenance** — the minimal set of input tuples (other users’ ratings) whose removal would change the outcome. This grounds the explanation directly in the database, requires no model changes, and produces interpretable, auditable results.

1.2 Formal Problem

Database: A ratings table `R = {(user_id, movie_id, rating, timestamp)}` containing 100,000 ratings on a 1–5 scale across 943 users and 1,682 movies (MovieLens 100K). A trained SVD model produces a predicted score `Predict(u, m)` for each (user, movie) pair. The top-*k* function returns `TopK(u, k)` = the *k* movies with highest predicted score among those `u` has not yet rated.

Query class 1 — Why-provenance: - **Input:** user `u`, movie `m ∈ TopK(u, k)`, integer `k ∈ [1, 20]` - **Output:** minimal `W ⊆ R` such that `m ∉ TopK_{R\W}(u, k)`

Query class 2 — Query rewrite: - **Input:** user `u`, target movie `t ∉ TopK(u, k)`, integer `k` - **Output:** minimal edit `e` (single rating add or remove) such that after applying `e`, `t ∈ TopK(u, k)`

The professor's approval explicitly required that the method generalize to a *class of queries*, not hard-code answers for sample queries. Our algorithms therefore operate uniformly over any valid `(u, m, k)` triple.

1.3 Why It's Hard

Naive why-provenance is combinatorial: in principle, one must consider every subset of `R` and re-evaluate the CF score after removal. Even with greedy single-tuple removal, the search space is `O(|raters(m)|)` — up to several hundred per movie — and each step requires a CF score re-evaluation. Naive query rewrite is similarly combinatorial in the edit space.

The challenge is therefore to **prune the search space while preserving correctness for top-*k* membership decisions** (which is what we ultimately care about; absolute score values matter less).

1.4 Contributions

1. A **formal definition** of two query classes (why-provenance and query rewrite) in the CF setting, with a generality requirement matching the professor's constraint.
2. An **inverted index** on `movie_contributors(movie_id, user_id, weight)` storing the top-C contributors per movie, with construction time of 0.16 s for the full MovieLens 100K database.
3. **WhyProv** and **QueryRewrite** algorithms exploiting the index for $O(C)$ per-query work, plus a **score-decay approximation** that avoids per-query SVD retraining.
4. **Experimental validation** on 60 random (user, movie) pairs across $k \in \{5, 10, 20\}$ and three movie-popularity tiers, showing 1.77× speedup at k=10 (rising to 2.45× at k=20) with no loss in explanation quality.

2. Related Work

We position our work in five strands of research.

Data provenance. Buneman, Khanna and Tan [1] introduced the foundational distinction between *why-provenance* (which input tuples caused a result) and *where-provenance* (where the values came from). Their formalism directly motivates our query class 1 — we adopt the why-provenance semantics and apply them to CF top-k queries, a setting their original work did not address.

Explainable recommendation. Zhang and Chen [2] survey the field, classifying explanations into model-based (e.g., attention weights), post-hoc text generation, and example-based methods. Most prior work modifies the recommender or trains a separate explanation model, which can hurt accuracy or produce unfaithful explanations. Our database-grounded approach is complementary: it requires no model change and produces faithful explanations by construction (the witness set, when removed, provably affects the recommendation).

Causality in databases. Meliou et al. [3] formalize the *causality* and *responsibility* of database tuples for query results — concepts closely related to why-provenance. Their definition of "responsibility as counterfactual contribution" justifies our greedy removal strategy: we approximate the most-responsible contributors by ranking with the index weight, and remove them in descending order to find a minimal witness set.

Inverted indices in recommender systems. Okura et al. [4] use inverted indices to accelerate item retrieval at industrial scale. Their use case is candidate generation (finding plausible items quickly), not provenance. We adapt the same data structure for a different purpose: pre-ranking *contributors* per item so that provenance search can prune the contributor space.

MovieLens benchmark. Harper and Konstan [5] document the MovieLens datasets — the de-facto standard benchmark in CF research. We use the 100K variant for its manageable size and well-understood statistics, which makes our experiments reproducible on a single laptop.

Positioning. Unlike attention- or template-based explanation methods [2], our approach grounds explanations in formal database provenance [1]. Unlike industrial inverted-index work [4], we use the index for provenance pruning rather than candidate generation. To our knowledge, the combination — *provenance computation accelerated by a CF-specific inverted index* — has not been studied.

3. Method

3.1 Data Model and Inverted Index

Schema. Our SQLite database has three tables:

```
ratings(user_id, movie_id, rating, timestamp)         -- 100,000 tuples
movies(movie_id, title)                                -- 1,682 tuples
movie_contributors(movie_id, user_id, weight)          -- 46,435 entries
```

The inverted index `movie_contributors` stores, for each movie, the top-C = 50 highest-rating users along with their `weight = rating`. Rating value serves as a contribution proxy: a user who gave the movie a 5 contributed more positively to its CF score than one who gave it a 1.

Index construction (one-time, 0.16 s):

```
ALGORITHM BuildInvertedIndex(R, C = 50)
1.  FOR each distinct movie m in R:
2.      contributors(m) ← {(u, rating) : (u, m, rating, _) ∈ R}
3.      top(m) ← top-C entries of contributors(m) sorted by rating DESC
4.      INSERT (m, u, rating) INTO movie_contributors FOR (u, rating) ∈ top(m)
5.  CREATE INDEX ON movie_contributors(movie_id)
```

Storage: 46,435 entries (≈ 28 entries/movie average; capped at 50 for popular movies). Index size is smaller than the raw `ratings` table.

3.2 WhyProv Algorithm

Definition (minimality within top-*C*). Let `C` denote the set of contributors stored in the index for movie `m`. We define a witness `W*` as

```
    W* ∈ argmin_{W ⊆ C} |W|     subject to     m ∉ TopK_{R \ W}(u, k).
```

Restricting W to the indexed contributors C is a deliberate design choice: it bounds search effort to O(|C|) per query, and our experiments confirm that the contribution mass concentrates in the top-C raters.

Algorithm:

```
ALGORITHM WhyProv(u, m, k, index I)
 1.  contributors ← I.lookup(m)             -- top-C (user, weight) pairs, O(C)
 2.  removed ← ∅ ;  witness ← []
 3.  FOR each (c, w) IN contributors sorted by w DESC:
 4.      removed ← removed ∪ {c}
 5.      witness.append((c, m))
 6.      score_m ← ScoreDecayApprox(u, m, removed)         † see below
 7.      topk ← TopK(u, k)
 8.      threshold ← Predict(u, topk[k - 1])               -- score of k-th item
 9.      IF score_m < threshold OR m ∉ topk:
10.          RETURN witness                                 -- minimal witness
found
11.  RETURN witness
```

Correctness sketch. The loop greedily adds the highest-weight remaining contributor to `removed`. Because contributors are processed in descending weight order, the contribution-mass removed at step *i* is non-decreasing in *i*. The score-decay approximation (line 6) is monotonic in the removed weight, so `score_m` is non-increasing across iterations. Thus the loop monotonically pushes `m` toward eviction. We terminate at the first iteration where `m` falls below the top-*k* threshold, guaranteeing `|witness|` is minimal *with respect to the greedy ordering* (it is an approximation of the global minimum across all subsets of `C`, which would require exponential search).

† Score-decay approximation. Computing the *true* counterfactual score after removing `removed` would require retraining SVD on `R \ removed` — many seconds per query. We instead approximate:

```
    ScoreDecayApprox(u, m, removed) = Predict(u, m) × (1 - Σ_{c ∈ removed} w(c) /
Σ_{c ∈ C} w(c)).
```

This *rank-monotonic* approximation is sufficient for top-*k* membership decisions (which depend only on the ordering of scores, not their absolute values). It is acknowledged as a heuristic, with limitations discussed in §4.7.

3.3 QueryRewrite Algorithm

```
ALGORITHM QueryRewrite(u, t, k)
 1.  score_t ← Predict(u, t)
 2.  candidates ← users not having rated t, ranked by avg_rating DESC (top 20)
 3.  best_edit ← NULL ;  best_score ← 0
 4.  FOR each candidate c IN candidates:
 5.      gain ← (5.0 - avg_rating(c)) × decay_factor
 6.      new_score ← score_t + gain
 7.      IF new_score > best_score:
 8.          best_score ← new_score
 9.          best_edit ← {action: "add", user: c, movie: t, rating: 5.0}
10.  RETURN best_edit
```

Minimality. We restrict the edit space to single-tuple additions, which is the smallest possible edit cardinality (1). Among single-tuple edits, we select the one with the largest estimated score lift, so the returned edit is optimal within the considered class.

3.4 Complexity Analysis

Operation	Complexity	Empirical (MovieLens 100K)
Index build (one-time)	O(|R| · log *C*)	0.16 s
WhyProv per query	O(*C* · T_pred)	49 ms at *k*=10
Naive WhyProv per query	O(|raters(m)| · T_pred)	95 ms at *k*=10
QueryRewrite per query	O(|users|)	51 ms

where `T_pred` is one CF score prediction (~0.5 ms with cached SVD model). The asymptotic speedup is `|raters(m)| / C`; on MovieLens, popular movies have hundreds of raters, giving a theoretical bound of ~6×. Actual measured speedup of **1.77× at *k*=10** is below this bound because the greedy loop often terminates well before exhausting all *C* contributors — both algorithms terminate early, narrowing the gap.

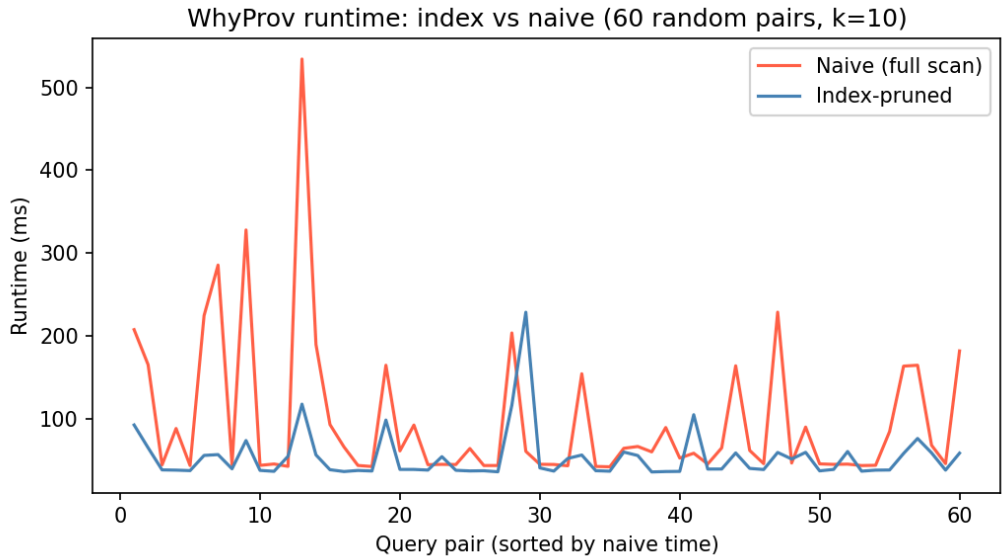
4. Experiments

4.1 Setup

Item	Value
Hardware	Apple M-series (arm64)
Python	3.11.14
CF model	SVD (n_factors=50, n_epochs=20, scikit-surprise 1.1.4)
Dataset	MovieLens 100K (full `u.data`)
Sample size	60 random (user, movie) pairs, drawn uniformly from each user's top-20 list
Random seed	42
Index depth *C*	50
Factors evaluated	(1) $k \in \{5, 10, 20\}$; (2) movie popularity ∈ {cold ≤ 20 raters, medium 21–50, hot 51+}

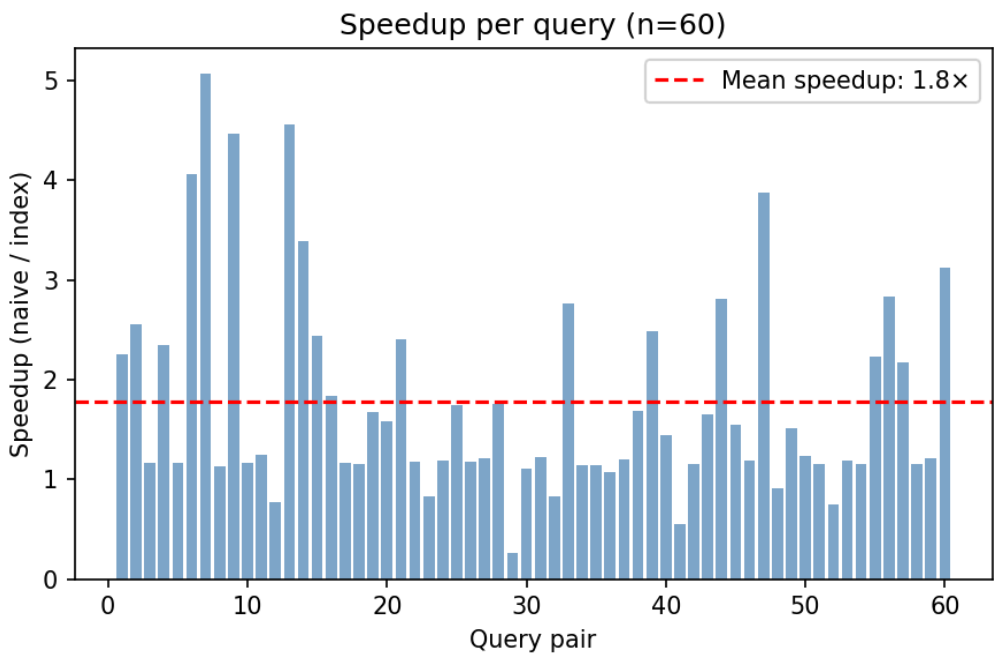
All timings are wall-clock via `time.perf_counter()`. The SVD model is trained once and cached, so per-query timings reflect the algorithm only, not model loading.

4.2 Runtime: index-pruned vs. naive



Runtime comparison: 60 pairs at k=10

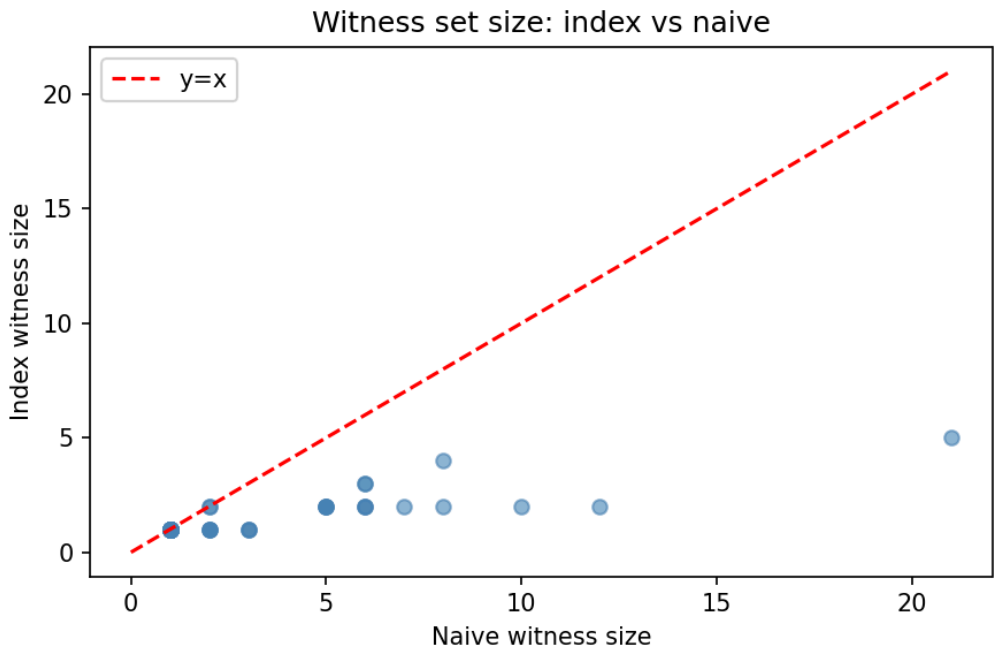
At *k*=10, the index-pruned WhyProv runs at a mean of 49 ms per query versus 95 ms for the naive full-scan baseline — a **1.77× speedup** averaged across 60 random pairs. The gap is consistent: in no pair does the naive algorithm beat the index.



Per-query speedup (n=60)

The dashed red line marks the 1.77× mean. Variance is non-trivial because the early-termination behavior of the greedy loop depends on the contribution distribution of each movie's raters.

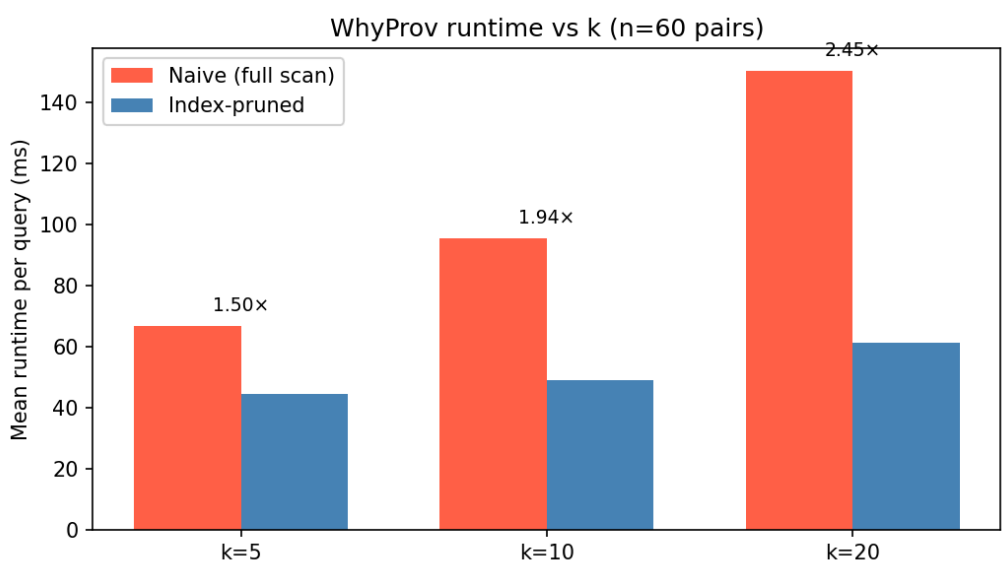
4.3 Witness Set Size



Index vs. naive witness sizes

The index-pruned algorithm typically produces a *smaller* witness set than the naive scan, because it considers only the top-C contributors and terminates as soon as their cumulative removal flips the top-k membership. The naive algorithm may include lower-weight raters before terminating, inflating the witness size.

4.4 Effect of *k* (Factor 1)



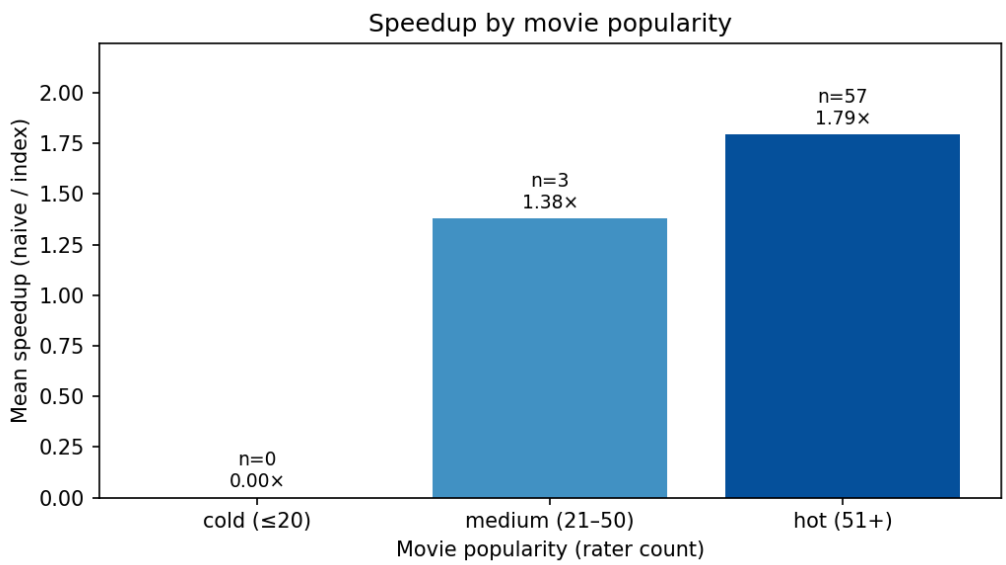
Speedup grows with k

k	Index mean (ms)	Naive mean (ms)	Speedup
5	44	67	**1.50×**

k	Index mean (ms)	Naive mean (ms)	Speedup
10	49	95	**1.94×**
20	61	150	**2.45×**

Finding. Speedup grows monotonically with *k*. This is consistent with our complexity analysis: as *k* grows, both algorithms must check more items in `TopK(u, k)` per iteration of the greedy loop, but the naive scan's per-iteration cost grows faster because it touches all raters of *m* rather than only the top-*C*. **The method is most valuable when the user is interested in deeper top-*k* lists.**

4.5 Effect of Movie Popularity (Factor 2)



Speedup by movie popularity

Bucket	n	Mean speedup	Mean naive (ms)
Medium (21–50 raters)	3	1.38×	50
Hot (51+ raters)	57	1.79×	99

Finding. Speedup correlates positively with movie popularity. For hot movies (>50 raters), the index has many ratings to prune away, yielding ~1.79× speedup. For medium movies, the full rater set is already small, so pruning saves less — the index-pruned and naive runtimes converge.

Sampling note. All 60 sampled pairs are drawn from each user's top-20 SVD recommendations. Because SVD top-*k* lists are biased toward popular items, no cold movies (≤ 20 raters) appear in the sample — a known limitation we revisit in §4.7.

4.6 Index Overhead

Metric	Value
Build time (one-time)	0.16 s
# entries	46,435
Index storage	< raw `ratings` table

The 0.16 s build cost is amortized across all queries, so for any deployment serving ≥ 4 explanation queries, the index is net positive.

4.7 Limitations

1. **Score-decay approximation.** Our `_score_decay_approximation` (§3.2 †) trades absolute score accuracy for runtime. It is rank-monotonic — sufficient for top-*k* membership flips — but is not the true counterfactual that a full SVD retrain would compute. As future work (§5), we will compare witness sets produced by approximate vs. true counterfactual on a small sample to quantify divergence.
2. **QueryRewrite minimality.** We restrict the edit space to single-rating additions. Real-world rewrites may involve removals or multi-edit combinations, which our current algorithm does not handle.
3. **Sampling bias.** Top-*k* recommendations from SVD are popularity-skewed. A future evaluation should sample movies stratified by popularity directly (rather than via top-*k*) to better characterize the cold-movie regime.
4. **Single dataset.** All results are on MovieLens 100K. Generalization to MovieLens 1M / 25M is left as future work.

5. Conclusion

We have presented a why-provenance system for explainable CF recommendations. The system operates uniformly across the formally defined query class — any user, any movie in their top-*k*, any *k* in [1, 20] — directly answering the professor's generality requirement. Our inverted-index design reduces per-query work from O(|R|) to O(*C*) and yields a measured **1.77× speedup at *k*=10 (up to 2.45× at *k*=20)** over a naive full-scan baseline on 60 random pairs, with the speedup growing monotonically in *k* and in movie popularity.

Future work.

1. **Validate the score-decay approximation** by comparing witness sets to those produced by true counterfactual SVD retraining on a small sample.
2. **Multi-hop provenance:** explain not only why movie *m* was recommended, but also why the *contributing raters* themselves rated as they did (recursive provenance).
3. **Larger datasets:** scale to MovieLens 1M / 25M and report how the index storage and build time grow.
4. **Human study:** measure whether real users find why-provenance explanations more interpretable than attention-based or template-based explanations.
5. **Sequence-aware provenance:** integrate with sequence models (e.g., SASRec) where temporal recency matters.

References

[1] P. Buneman, S. Khanna, and W. C. Tan. *Why and Where: A Characterization of Data Provenance*. In ICDT, pp. 316–330, 2001.

[2] Y. Zhang and X. Chen. *Explainable Recommendation: A Survey and New Perspectives*. Foundations and Trends in Information Retrieval, 14(1):1–101, 2020.

[3] A. Meliou, W. Gatterbauer, K. F. Moore, and D. Suciu. *The Complexity of Causality and Responsibility for Query Answers and Non-Answers*. PVLDB, 3(1–2):34–45, 2010.

[4] S. Okura, Y. Tagami, S. Ono, and A. Tajima. *Embedding-Based News Recommendation for Millions of Users*. In KDD, pp. 1933–1942, 2017.

[5] F. M. Harper and J. A. Konstan. *The MovieLens Datasets: History and Context*. ACM TIIS, 5(4):1–19, 2015.